

Universidad Argentina de la Empresa
Facultad de Ingeniería y Ciencias Exactas



Documento de Definición del Proyecto - CostaApp

Alumno:

Ramiro Zayas Contreras - razayas@uade.edu.ar - 1169993

Carrera:

Licenciatura en Gestión de Tecnología de la Información

Profesor:

Gladkoff, Nicolas Alejandro - ngladkoff@uade.edu.ar - 1085075

Bianca, Nahuel - nbianca@uade.edu.ar - 1223653

Fecha de entrega:

22 de abril de 2026

Registro de Cambios — Versión 2.0

Nº	Área Modificada	Descripción del Cambio	Justificación Técnica
C01	Fuente de Datos (Secs. 1, 2, 4)	Reemplazo de la API pública de Mercado Libre por una API REST Propia (Mock API estática) hospedada en la nube (Mocky.io / JSONBin).	Elimina dependencias de OAuth de terceros, bloqueos por CORS y riesgos de web scraping, garantizando disponibilidad controlada del backend durante el ciclo de vida del proyecto.
C02	Persistencia Local — Room (Secs. 2, 3)	Redefinición del rol de Room: de "Única Fuente de Verdad total" a Caché Inteligente (Smart Cache) que almacena únicamente las propiedades visualizadas recientemente y los favoritos del usuario.	Reduce el consumo de almacenamiento local, mejora la escalabilidad y elimina la sincronización innecesaria de todo el catálogo en dispositivos con recursos limitados.
C03	Integración de IA — Gemini SDK (Secs. 2, 4)	Eliminación del CU08 "Asistente de Búsqueda Inteligente". Reemplazo por el nuevo CU08: Enriquecimiento de Contenido con IA (Resumen Inteligente), que genera automáticamente un análisis "2 Ventajas / 1 Desventaja" en la pantalla de detalle de cada propiedad.	El caso de uso anterior presentaba alta complejidad de implementación y dependencia de conectividad. El nuevo CU es determinista, acotado y de mayor valor percibido para el usuario final.

C04	Firebase Firestore (Secs. 3)	Eliminación de la colección user_favorites con sub-colecciones complejas. Firestore queda reducida a una única colección users con un campo favorite_property_ids (array de strings).	Minimiza el costo de operaciones de lectura/escritura en Firestore, simplifica las reglas de seguridad y convierte la nube en un sincronizador liviano entre dispositivos en lugar de un repositorio de datos pesado.
-----	---------------------------------	---	---

1. Nombre y Objetivo: Propuesta de Valor

Nombre del proyecto: CostaApp

1.1 Objetivo General

El propósito de este proyecto es el diseño e implementación integral de una aplicación móvil nativa bajo la plataforma Android, desarrollada utilizando Kotlin como lenguaje núcleo. CostaApp surge como una solución tecnológica centralizada para la búsqueda y gestión de propiedades (ventas y alquileres) en la Costa Atlántica, con foco inicial en la ciudad de Pinamar. Para garantizar la disponibilidad y el control total sobre la fuente de datos, la aplicación consumirá datos a través de una API REST Propia (Mock API estática) orientada al rubro inmobiliario, hospedada en la nube mediante servicios como Mocky o JSONBin. Esta decisión arquitectónica aísla la aplicación de bloqueos por CORS, flujos de autenticación de terceros (OAuth) y riesgos asociados al web scraping de plataformas comerciales. El desarrollo se alinea con los estándares de la industria, priorizando la escalabilidad y la robustez arquitectónica mediante el uso de arquitecturas distribuidas y modernas interfaces declarativas.

1.2 Propuesta de Valor

- La aplicación busca transformar la experiencia de búsqueda inmobiliaria local mediante una interfaz moderna en Jetpack Compose, priorizando la eficiencia en el ciclo de vida de los componentes mediante State Hoisting y el manejo de estados reactivos. Los pilares que sostienen el valor de la propuesta, integrando los requisitos técnicos exigidos, son:
- Identidad y Acceso Profesional: Implementación de una transición fluida mediante la API de Splash de Android y un sistema de autenticación vía Firebase (Google Auth) con persistencia de sesión automática.
- Disponibilidad Ininterrumpida (Smart Cache): La aplicación implementa un Caché Inteligente mediante Room que almacena localmente las últimas propiedades visualizadas y los favoritos del usuario. Esto garantiza la consulta de contenido relevante vía Retrofit incluso en zonas de baja conectividad, sin necesidad de sincronizar todo el catálogo.
- Arquitectura de Alta Calidad: Se asegura la mantenibilidad del código a través del patrón MVVM con una separación de capas estricta, mediada por el Repository Pattern y la inyección de dependencias para el desacoplamiento de componentes.
- Experiencia de Búsqueda Dinámica: Visualización de datos en tiempo real mediante un listado dinámico (LazyColumn) que incluye un sistema de búsqueda y filtrado reactivo sobre la oferta inmobiliaria.
- Renderizado Multimedia Avanzado: Gestión eficiente de fotografías de propiedades mediante la librería Glide, aplicando transformaciones y placeholders según el estado de la carga dinámica.
- Asesoría Inteligente (AI SDK): Integración del Gemini Pro SDK para el enriquecimiento automático del contenido en la pantalla de detalle, generando un análisis conciso de ventajas y desventajas de cada propiedad.

- Sincronización en la Nube (Liviana): Gestión remota de preferencias del usuario mediante Firebase Firestore, actuando exclusivamente como sincronizador de favoritos entre dispositivos.
- Servicios de Mapas: Integración de Google Maps SDK for Android para la visualización geolocalizada de la oferta inmobiliaria y navegación interactiva por el ejido urbano de Pinamar.

2. Casos de Uso

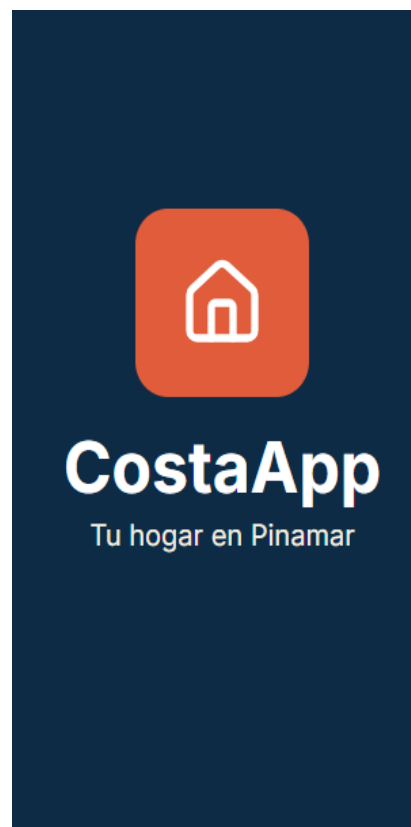
2.1 Introducción

Esta sección detalla el comportamiento de **CostaApp** mediante Casos de Uso (CU) siguiendo los estándares de **Alistair Cockburn**. Se prioriza la arquitectura **Offline-first**, donde la base de datos local (**Room**) actúa como la única fuente de verdad (SSOT). Se describen las interacciones, los estados de la interfaz (Loading, Success, Error) y el flujo de datos con la API de **Mercado Libre** y **Firebase**.

2.2 Especificación de Casos de Uso

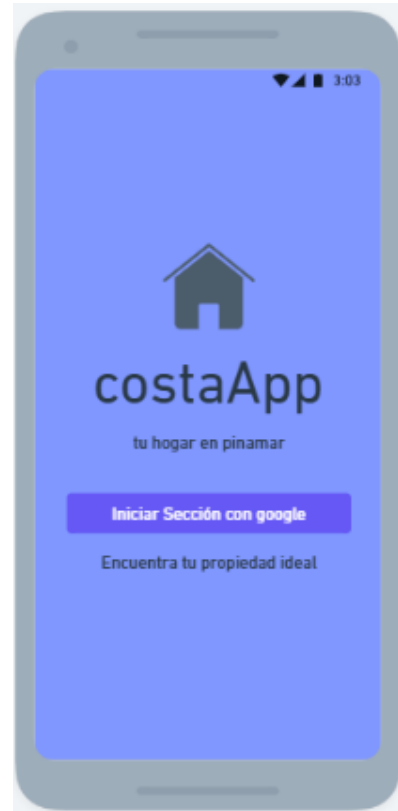
CU01: Inicialización y Transición (Splash Screen)

- **Identificador:** CU01.
- **Actor:** Usuario.
- **Descripción:** El sistema presenta la identidad de marca mientras valida el estado de la sesión.
- **Precondiciones:** La aplicación es ejecutada por el usuario.
- **Flujo Principal:**
 1. El sistema inicia la API de Splash de Android.
 2. El sistema verifica mediante el SDK de Firebase si existe una sesión activa.
 3. El sistema resuelve la navegación: si hay sesión, dirige al Listado; de lo contrario, al Login.
- **Flujos Alternos:** N/A.
- **Postcondiciones:** El sistema muestra la pantalla de destino correspondiente al estado de autenticación.
- **Detalle de Datos:** Recupera el objeto `FirebaseUser (UID)`.



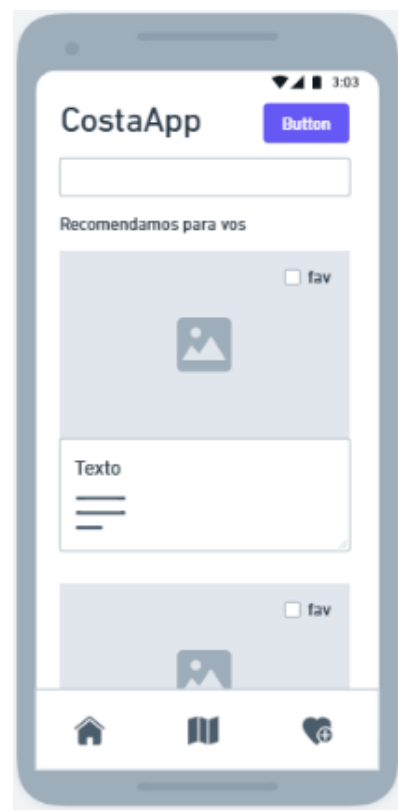
CU02: Autenticación de Usuario (Firebase Auth)

- **Identificador:** CU02.
- **Actor:** Usuario.
- **Descripción:** Permitir el acceso seguro mediante cuentas Gmail.
- **Precondiciones:** El usuario se encuentra en la pantalla de Login y no posee sesión activa.
- **Flujo Principal:**
 1. El usuario presiona el botón "Ingresar con Google".
 2. El sistema invoca el flujo nativo de autenticación de Firebase.
 3. El usuario selecciona su cuenta y confirma la identidad.
 4. El sistema valida las credenciales y persiste la sesión.
- **Flujos Alternos:** El usuario cancela el diálogo; el sistema permanece en la pantalla de Login.
- **Postcondiciones:** El usuario queda autenticado y es redirigido al flujo principal.
- **Detalle de Datos:** Recibe del SDK de Firebase: DisplayName, Email y PhotoURL.



CU03: Consulta y Sincronización de Propiedades (Offline-first)

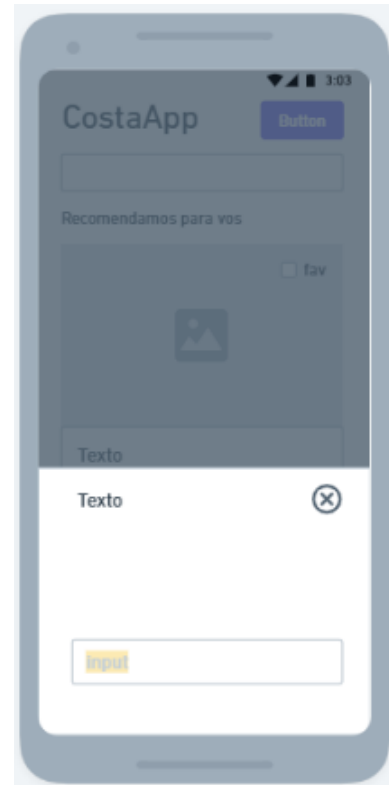
- **Identificador:** CU03.
- **Actor:** Usuario.
- **Descripción:** Recuperar la oferta inmobiliaria desde la API REST Propia, almacenando en caché local únicamente las propiedades visualizadas recientemente para garantizar disponibilidad sin conexión.
- **Precondiciones:** El usuario ha ingresado a la pantalla principal.
- **Flujo Principal:**
 1. El sistema solicita el listado de propiedades a la API REST Propia vía Retrofit.
 2. El sistema recibe el JSON y actualiza el Caché Inteligente en Room con las propiedades del listado actual.
 3. El sistema renderiza el listado observando el estado actualizado de la caché local.



- **Flujos Alternos:** Sin conexión a internet; el sistema omite los pasos 1 y 2, cargando únicamente las propiedades previamente cacheadas en Room.
- **Postcondiciones:** El usuario visualiza la lista actualizada de propiedades en Pinamar.
- **Detalle de Datos:** JSON Response: results (id, title, price, zone, operationType).

CU04: Búsqueda y Filtrado Reactivo

- **Identificador:** CU04.
- **Actor:** Usuario.
- **Descripción:** Localizar propiedades específicas mediante criterios de búsqueda sobre la caché local.
- **Precondiciones:** El listado de propiedades se encuentra visible en pantalla.
- **Flujo Principal:**
 1. El usuario ingresa texto en la barra de búsqueda (ej: "Bunge").
 2. El ViewModel procesa el filtro sobre el estado de la lista obtenida de Room.
 3. La UI se recompone automáticamente mostrando solo los resultados coincidentes.
- **Flujos Alternos:** No hay coincidencias; el sistema muestra un mensaje de "No se encontraron resultados".
- **Postcondiciones:** El listado refleja únicamente los elementos filtrados.



CU05: Visualización de Detalle de Inmueble

- **Identificador:** CU05.
- **Actor:** Usuario.
- **Descripción:** Consultar información técnica profunda y fotografías de una propiedad.
- **Precondiciones:** El usuario selecciona una propiedad del listado dinámico.
- **Flujo Principal:**
 1. El sistema navega a la pantalla de Detalle transfiriendo el ID de la propiedad.
 2. El sistema consulta los datos completos de la propiedad en Room (Caché Inteligente).
 3. El sistema carga las imágenes dinámicamente mediante Glide.
 4. El sistema invoca el SDK de Gemini Pro para generar el resumen inteligente (ver CU08)
- **Flujos Alternos:** Error al cargar la imagen; Glide muestra un placeholder de error.



- **Postcondiciones:** El sistema presenta la ficha completa del inmueble con el análisis de IA integrado.
- **Detalle de Datos:** Objeto PropertyDetail (latitude, longitude, rooms, surface, zone, price).

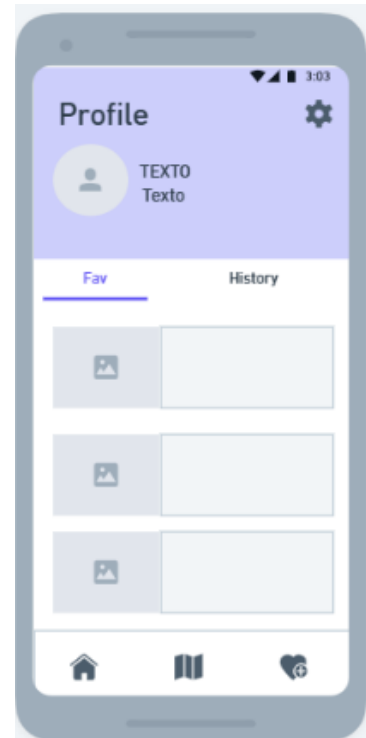
CU06: Registro de Propiedad Favorita

- **Identificador:** CU06.
- **Actor:** Usuario.
- **Descripción:** Marcar una propiedad como favorita, persistiendo el ID localmente en Room y sincronizando el listado de IDs en Firestore para disponibilidad multi-dispositivo.
- **Precondiciones:** El usuario visualiza el detalle de una propiedad y está autenticado.
- **Flujo Principal:**
 1. El usuario presiona el botón de "Favorito".
 2. El sistema actualiza el flag isFavorite en la entidad local de Room.
 3. El sistema actualiza el array favorite_property_ids en el documento del usuario en Firebase Firestore.
 4. El sistema confirma el guardado visualmente (cambio de icono).
- **Flujos Alternos:** Error de escritura en Firestore; el sistema mantiene el cambio local y reintentar la sincronización en la próxima sesión con conexión activa.
- **Postcondiciones:** El ID de la propiedad queda registrado localmente y sincronizado en la nube.
- **Detalle de Datos:** propertyId (String) sincronizado en el campo favorite_property_ids del documento de usuario en Firestore.



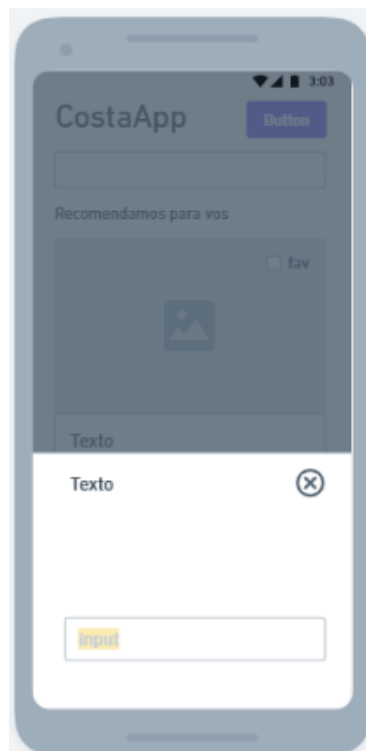
CU07: Consulta de Favoritos en Modo Offline

- **Identificador:** CU07.
- **Actor:** Usuario.
- **Descripción:** Acceder a la lista de propiedades marcadas como favoritas, almacenadas en el Caché Inteligente local.
- **Precondiciones:** El usuario accede a la sección "Favoritos".
- **Flujo Principal:**
 1. El sistema consulta en Room todas las entidades con isFavorite = true.
 2. El sistema filtra y renderiza el listado de propiedades favoritas en pantalla.
- **Flujos Alternos:** Caché vacía; el sistema sugiere explorar nuevas propiedades.
- **Postcondiciones:** El usuario visualiza sus propiedades guardadas independientemente de la conectividad.
- **Detalle de Datos:** Query SQL en Room sobre la entidad Property con filtro WHERE isFavorite = 1.



CU08: Enriquecimiento de Contenido con IA (Resumen Inteligente)

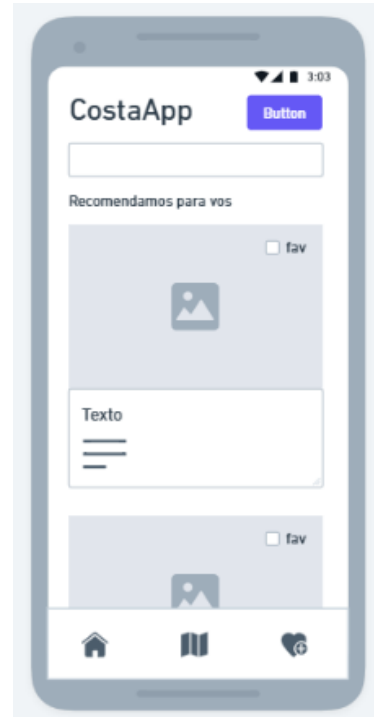
- **Identificador:** CU08.
- **Actor:** Usuario.
- **Descripción:** Al ingresar a la pantalla de detalle de una propiedad, el sistema utiliza el SDK de Gemini Pro para analizar los datos estructurados del inmueble (ambientes, superficie en m², zona, precio y tipo de operación) y generar automáticamente un texto breve con 2 Ventajas y 1 Desventaja de la propiedad, actuando como un asesor inmobiliario virtual.
- **Precondiciones:** El usuario ha navegado al detalle de una propiedad y el dispositivo cuenta con conexión a internet.
- **Flujo Principal:**
 1. El sistema extrae los atributos clave de la propiedad desde Room: rooms, surface, zone, price, operationType.
 2. El sistema construye un prompt estructurado y lo envía al Gemini Pro SDK.
 3. La IA procesa los datos y retorna un texto con el formato: " Ventaja 1 / Ventaja 2 / Desventaja 1".



4. El sistema renderiza el análisis en un componente destacado dentro de la pantalla de detalle.
- **Flujos Alternos:** Sin conexión o falla del SDK; el sistema omite el componente de IA y muestra únicamente los datos crudos de la propiedad.
 - **Postcondiciones:** El usuario dispone de un análisis contextualizado de la propiedad para facilitar la toma de decisiones.
 - **Detalle de Datos:** Input (Gemini SDK): objeto estructurado con atributos de la propiedad / Output: String con formato "Ventajas / Desventaja".

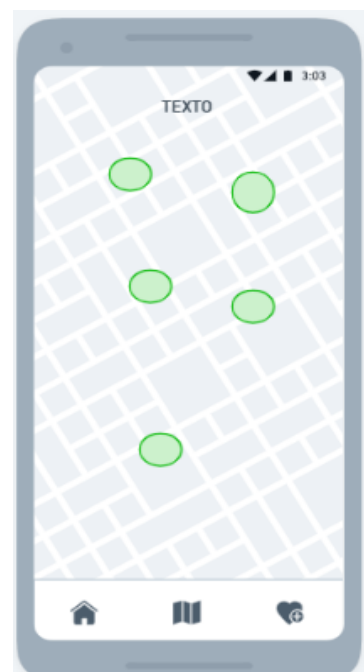
CU09: Gestión de Estados de Error y Conexión

- **Identificador:** CU09.
- **Actor:** Sistema.
- **Descripción:** Informar al usuario sobre fallos técnicos o de red de forma explícita.
- **Precondiciones:** Ocurre una excepción durante una operación de red o base de datos.
- **Flujo Principal:**
 1. El sistema detecta el fallo (ej: Timeout de la API REST Propia).
 2. El ViewModel emite un estado de UI tipo "Error".
 3. La UI renderiza un componente informativo con opción de "Reintentar".
- **Flujos Alternos:** El usuario presiona "Reintentar"; el sistema vuelve a ejecutar la acción fallida.
- **Postcondiciones:** El usuario está informado del estado actual de la aplicación.
- **Detalle de Datos:** Objeto capturado por el sistema.



CU10: Localización Geográfica en Mapa

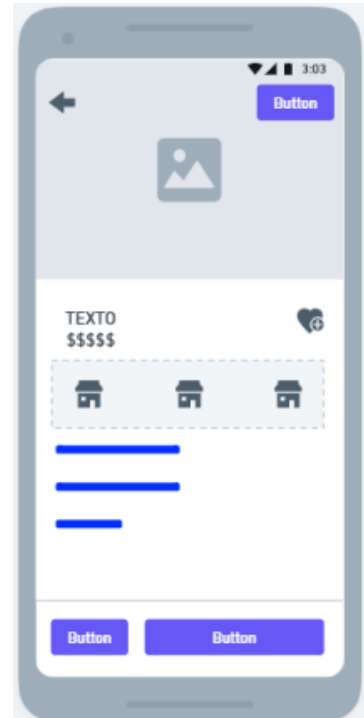
- **Identificador:** CU10.
- **Actor:** Usuario.
- **Descripción:** Visualizar propiedades en un mapa interactivo para mejorar la orientación.
- **Precondiciones:** El usuario se encuentra en la pantalla de Listado o Detalle.
- **Flujo Principal:**
 1. El usuario selecciona la opción "Ver en Mapa".
 2. El sistema carga Google Maps e integra los marcadores basados en las coordenadas de Room.
 3. El sistema centra el mapa en la ubicación de la propiedad o zona seleccionada.



- **Flujos Alternos:** Sin conexión; el sistema muestra los marcadores sobre el mapa en caché.
- **Postcondiciones:** El usuario visualiza la ubicación espacial de los inmuebles.
- **Detalle de Datos:** Coordenadas latitude y longitude recuperadas de Room.

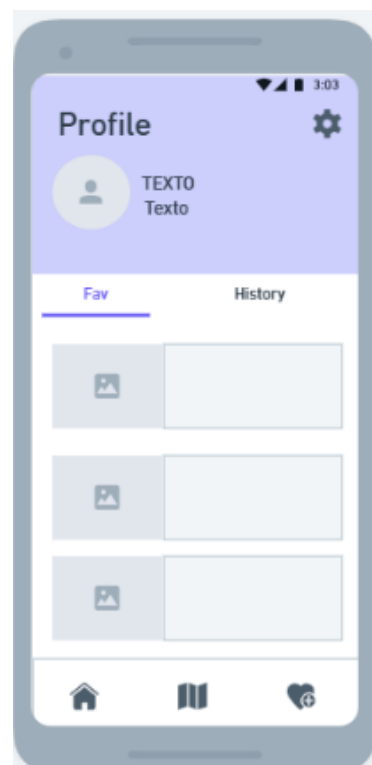
CU11: Contacto Directo con el Anunciante

- **Identificador:** CU11.
- **Actor:** Usuario.
- **Descripción:** Canal de comunicación inmediato con la inmobiliaria responsable.
- **Precondiciones:** El usuario se encuentra visualizando el detalle de una propiedad.
- **Flujo Principal:**
 1. El usuario presiona el botón "Contactar por WhatsApp".
 2. El sistema utiliza un **Intent** de Android para abrir la aplicación de mensajería.
 3. El sistema envía un mensaje predefinido con la información básica del inmueble.
- **Flujos Alternos:** WhatsApp no instalado; el sistema sugiere contacto vía Email.
- **Postcondiciones:** Se inicia una interacción externa al sistema.
- **Detalle de Datos:** String formateado con el `id` de la propiedad y su título.



CU12: Historial de Propiedades Visitadas

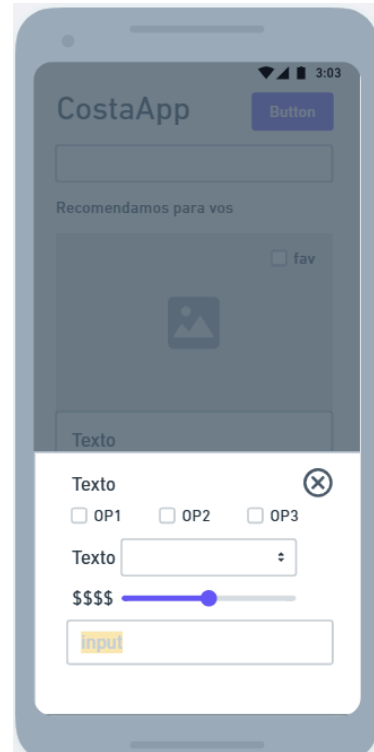
- **Identificador:** CU12.
- **Actor:** Usuario.
- **Descripción:** Retomar rápidamente las propiedades consultadas recientemente, disponibles en el Caché Inteligente local..
- **Precondiciones:** El usuario ha accedido al detalle de al menos una propiedad.
- **Flujo Principal:**
 1. El usuario accede a la sección "Recientes".
 2. El sistema consulta en Room las entidades con campo `lastVisit` no nulo.
 3. El sistema renderiza el listado ordenado cronológicamente de forma descendente.
- **Flujos Alternos:** Caché vacía; se muestra mensaje invitando a explorar.
- **Postcondiciones:** El usuario visualiza su trayectoria de navegación.



- **Detalle de Datos:** Lectura del campo lastVisit (Timestamp) en la tabla properties de Room. Lectura de campos lastVisit en la tabla Property.

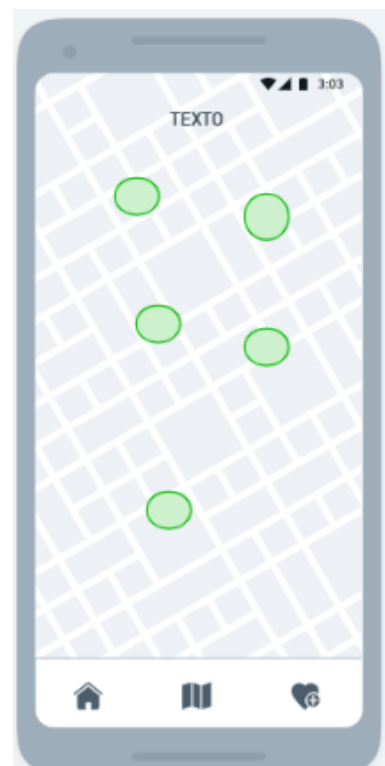
CU13: Ordenamiento Dinámico de Resultados

- **Identificador:** CU13.
- **Actor:** Usuario.
- **Descripción:** Reorganizar la presentación de las propiedades según precio o ambientes.
- **Precondiciones:** El listado dinámico de propiedades es visible.
- **Flujo Principal:**
 1. El usuario abre el menú de "Ordenar por".
 2. El usuario selecciona un criterio (ej: "Menor precio").
 3. El ViewModel actualiza la consulta sobre Room con el parámetro de ordenamiento seleccionado.
- **Flujos Alternos:** N/A.
- **Postcondiciones:** El listado se ajusta a la prioridad seleccionada.
- **Detalle de Datos:** Parámetro de ordenamiento (ORDER BY price ASC/DESC) aplicado al ViewModel.



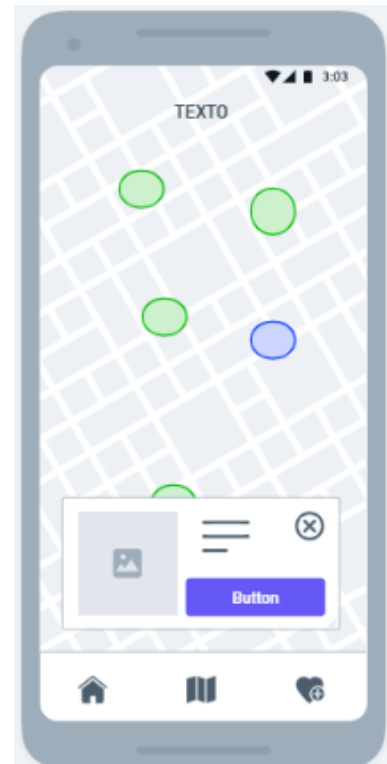
CU14: Exploración de Mapa (Pines y Globos)

- **Identificador:** CU14.
- **Actor:** Usuario.
- **Descripción:** Navegación libre por Pinamar visualizando la oferta mediante marcadores dinámicos.
- **Precondiciones:** El sistema ha cargado datos en el Caché Inteligente de Room.
- **Flujo Principal:**
 1. El usuario accede a la pestaña "Mapa".
 2. El sistema renderiza marcadores tipo "Globo" con el precio de cada inmueble cacheado.
 3. El usuario realiza scroll y zoom para explorar diferentes zonas de la ciudad.
- **Flujos Alternos:** Falla de Google Play Services; el sistema muestra el listado estándar.
- **Postcondiciones:** El usuario identifica espacialmente la oferta económica.



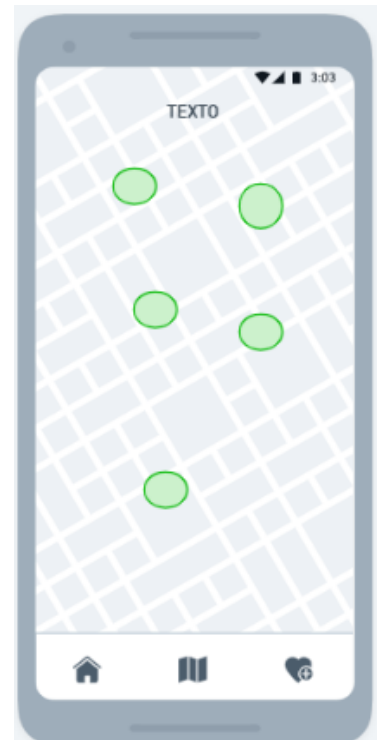
CU15: Previsualización de Propiedad desde Mapa (InfoWindow)

- **Identificador:** CU15.
- **Actor:** Usuario.
- **Descripción:** Obtener información rápida de un marcador sin abandonar el mapa.
- **Precondiciones:** El mapa está visible con marcadores cargados.
- **Flujo Principal:**
 1. El usuario presiona un marcador de precio.
 2. El sistema despliega una tarjeta con datos básicos del inmueble.
 3. El usuario presiona la tarjeta para navegar al detalle completo.
- **Flujos Alternos:** N/A.
- **Postcondiciones:** El usuario visualiza información preliminar contextualizada geográficamente.
- **Detalle de Datos:** Datos básicos de Property inyectados en el componente de mapa.



CU16: Filtrado por Radio de Cercanía

- **Identificador:** CU16.
- **Actor:** Usuario.
- **Descripción:** Filtrar propiedades que se encuentren dentro de un rango específico en el mapa.
- **Precondiciones:** El usuario está en la vista del mapa.
- **Flujo Principal:**
 1. El usuario activa el filtro "Cerca de mí" en el mapa.
 2. El ViewModel calcula las distancias basándose en la ubicación GPS y las coordenadas almacenadas en Room.
 3. La UI filtra los pines fuera del radio definido.
- **Flujos Alternos:** GPS desactivado; el sistema solicita activación de permisos.
- **Postcondiciones:** El mapa muestra únicamente inmuebles dentro del radio de cercanía.
- **Detalle de Datos:** Cálculo matemático de Haversine sobre coordenadas locales almacenadas en Room.



3. Diccionario de Datos

El Diccionario de Datos define la estructura lógica y física de la información gestionada por CostaApp. Se divide en dos componentes estratégicos: la persistencia local mediante Room (SQLite), que actúa como Caché Inteligente para lectura rápida y gestión de favoritos; y la persistencia remota mediante Firebase Firestore, que opera exclusivamente como sincronizador liviano de preferencias de usuario entre dispositivos.

3.1. Entidades en Room (Persistencia Local)

Room gestiona dos funciones principales: (1) almacenar las últimas propiedades visualizadas por el usuario para habilitar la navegación offline, y (2) persistir el estado de favoritos localmente. Los datos son mapeados desde el formato JSON de la API REST Propia hacia este esquema local.

3.1.1 Tabla: **properties**

Almacena la información técnica de los inmuebles, incluyendo coordenadas para el mapa y estados de interacción.

Campo	Tipo de Dato	Nulidad	Descripción	Restricciones
id	String	No Nulo	Identificador único de la propiedad en la Mock API.	Primary Key
title	String	No Nulo	Título descriptivo de la publicación.	-
description	String	No Nulo	Detalle extenso de las características del inmueble.	-
price	Double	No Nulo	Valor monetario del inmueble.	-
currency	String	No Nulo	Tipo de moneda (USD/ARS).	Long: 3
zone	String	No Nulo	Ubicación geográfica (Barrio o sector).	-

rooms	Integer	No Nulo	Cantidad de ambientes del inmueble.	Valor > 0
surface	Double	No Nulo	Superficie total en metros cuadrados (m²).	-
latitude	Double	No Nulo	Coordenada geográfica para renderizado de pines en Google Maps.	-
longitude	Double	No Nulo	Coordenada geográfica para renderizado de pines en Google Maps.	-
operationType	String	No Nulo	Tipo de operación: sale o rent.	-
imageUrl	String	No Nulo	URL de la imagen principal del inmueble (carga vía Glide).	Formato URL
isFavorite	Boolean	No Nulo	Flag local que indica si el usuario marcó la propiedad como favorita. Sincronizado con Firestore.	Default: false
lastVisit	Long	Nulo	Timestamp de la última vez que el usuario visualizó el detalle de esta propiedad (requerido por CU12).	-
lastUpdated	Long	No Nulo	Timestamp de la última sincronización con la API REST Propia.	-

3.2. Colecciones en Firebase Firestore (Persistencia Remota — Sincronizador Liviano)

Firestore actúa exclusivamente como sincronizador de preferencias de usuario entre dispositivos. Su estructura ha sido intencionalmente simplificada para minimizar el costo de operaciones de lectura/escritura en la nube y reducir la complejidad de las reglas de seguridad. La aplicación no almacena propiedades completas en Firestore.

3.2.1 Colección: **users**

Única colección en Firestore. Almacena el perfil básico del usuario y su lista de IDs de propiedades favoritas.

- **Document ID:** **uid** (Generado por Firebase Auth).
- **Campos:**
 - **uid (String):** Identificador único del usuario, coincidente con el UID de Firebase Auth.
 - **email (String):** Correo electrónico del usuario.
 - **displayName (String):** Nombre completo obtenido de la cuenta Google.
 - **photoUrl (String):** URL de la imagen de perfil de Google.
 - **createdAt (Timestamp):** Fecha de registro inicial en la aplicación.
 - **favorite_property_ids (Array<String>):** Listado de IDs de propiedades marcadas como favoritas. Actúa como sincronizador entre dispositivos: al iniciar sesión en un nuevo dispositivo, la aplicación lee este array y actualiza el flag `isFavorite` en Room para las propiedades presentes en la caché local.

La comunicación con el backend se realiza mediante la librería Retrofit, interactuando con la API REST Propia. Los datos se intercambian en formato JSON y son procesados por una capa de mapeo (Data Transfer Objects) antes de ser persistidos en el Caché Inteligente de Room.

4. Especificación de Endpoints

4.1. Listado de Propiedades en Pinamar

- Pantalla donde se utiliza: Listado Dinámico (Home) y Mapa.
- Descripción: Recupera el catálogo completo de inmuebles disponibles en Pinamar desde la Mock API estática.
- Verbo HTTP: **GET**.
- **URL:** `https://run.mocky.io/v3/{mock-id}/api/v1/properties/pinamar`
- Headers: **Accept:** `application/json`.

Estructura JSON (Response):

```
{
  "properties": [
    {
      "id": "COSTA-001",
      "title": "Casa moderna cerca de Av. Bunge",
      "price": 250000.0,
      "currency": "USD",
      "zone": "Centro",
      "rooms": 4,
      "surface": 180.0,
      "operationType": "sale",
      "thumbnail": "https://cdn.costaapp.com/images/COSTA-001.jpg",
      "latitude": -37.1089,
      "longitude": -56.8614
    },
    {
      "id": "COSTA-002",
      "title": "Departamento con vista al mar — Las Gaviotas",
      "price": 1500.0,
      "currency": "USD",
      "zone": "Las Gaviotas",
      "rooms": 2,
      "surface": 65.0,
      "operationType": "rent",
      "thumbnail": "https://cdn.costaapp.com/images/COSTA-002.jpg",
      "latitude": -37.0921,
      "longitude": -56.8530
    }
  ]
}
```

4.2. Detalle de Propiedad

- Pantalla donde se utiliza: Detalle del Elemento (CU05).
- Descripción: Obtiene los atributos técnicos completos de un inmueble específico, incluyendo coordenadas geográficas y galería de imágenes.
- Verbo HTTP: **GET**.
- URL: <https://run.mocky.io/v3/{mock-id}/api/v1/properties/{id}>
- Headers: **Accept: application/json**.

Estructura JSON (Response):

```
{
  "id": "COSTA-001",
  "title": "Casa moderna cerca de Av. Bunge",
  "description": "Amplia casa de 4 ambientes a 3 cuabras del mar. Cuenta con jardín, parrilla,
cochera cubierta y acabados de primera calidad.",
  "price": 250000.0,
  "currency": "USD",
  "zone": "Centro",
  "rooms": 4,
  "bathrooms": 3,
  "surface": 180.0,
  "operationType": "sale",
  "latitude": -37.1089,
  "longitude": -56.8614,
  "contactPhone": "+5492254000001",
  "pictures": [
    { "url": "https://cdn.costaapp.com/images/COSTA-001-01.jpg" },
    { "url": "https://cdn.costaapp.com/images/COSTA-001-02.jpg" }
  ]
}
```

4.3. Integración de Inteligencia Artificial (Gemini Pro SDK)

- Pantalla donde se utiliza: Detalle del Inmueble (CU08 — Enriquecimiento de Contenido).
- Descripción: La aplicación utiliza el Google AI SDK (Gemini Pro) de forma nativa para generar un análisis contextualizado de la propiedad visualizada.
- Funcionamiento:
- El sistema extrae los atributos estructurados de la propiedad desde Room: **rooms**, **surface**, **zone**, **price**, **operationType**.
- El sistema construye un prompt del tipo: *"Actúa como asesor inmobiliario. Dados los siguientes datos de una propiedad en Pinamar: [atributos], generará exactamente 2 ventajas y 1 desventaja en formato conciso."*
- El SDK procesa el prompt y retorna un texto estructurado con el análisis.
- El sistema renderiza el resultado en un componente destacado en la pantalla de Detalle.

4.4. Integración de Mapas (Google Maps SDK)

- Pantalla donde se utiliza: Mapa de Exploración (CU14) y Detalle de Propiedad (CU05).
- Descripción: Servicio de renderizado de mapas y gestión de marcadores (pines) interactivos.

- Origen de Datos: Las coordenadas (**latitude** y **longitude**) se obtienen del Caché Inteligente de Room, previamente sincronizado con la API REST Propia.
- Datos Recibidos: Teselas de mapas (imágenes de fondo) y metadatos de ubicación de Google Play Services.

Figma - Pantallas

Figma prototipo funcional (Funcional version 2):

<https://www.figma.com/make/O8lVgeRn5J8WsOqtPbGeHU/CostaApp---Prototipo?t=W1dX9KeyWFSAHd0s-1>

Prototipo de baja:

<https://whimsical.com/ramiro-student/64qbYb3vJCZgX7qNwreVWb>

GitHub – Repositorio

[razayas15/DA1-TPO-2026-CostaApp: Proyecto de aplicación móvil nativa en Kotlin para la gestión inmobiliaria en Pinamar. Trabajo Práctico Obligatorio - Desarrollo de Aplicaciones I \(UADE 2026\).](#)